

RELATÓRIO TÉCNICO FINAL - P2

Desafio 07: Detecção de Fraude em Transações Financeiras

Integrantes: Estevão Alves dos Santos (RA: 1990000)
Vanessa Kaori Kurauchi (RA: 2002344)

Disciplina: Machine Learning — Teoria e Aplicado

Professora: Ma. Nathália A. Lima

Grupo: 07

1. Introdução e Contextualização do Problema de Negócio

No setor financeiro e bancário atual, as fraudes em transações com cartões de crédito representam um desafio bilionário e constante. A sofisticação das organizações criminosas exige respostas em tempo real por parte das instituições financeiras para conter prejuízos operacionais imediatos e blindar a relação de confiança com o cliente final. O principal objetivo deste projeto é construir um modelo preditivo baseado em Machine Learning capaz de discriminar transações legítimas de tentativas de fraude.

A natureza desse problema configura um clássico cenário de **Classificação Binária Supervisionada Altamente Desbalanceada**. A classe minoritária (fraude) responde por uma fração ínfima do volume consolidado, o que inviabiliza o uso de métricas tradicionais como a acurácia global e requer um rigor metodológico extremo para evitar o vazamento de dados (*data leakage*) e falsas estimativas de sucesso.

2. Análise Exploratória de Dados (EDA)

O conjunto de dados utilizado é o célebre `creditcardfraud` (proveniente de estudos conjuntos do mlg-ulb e hospedado no Kaggle). A base compreende um volume histórico de transações reais ocorridas na Europa. Os resultados principais obtidos na fase de exploração indicam:

- **Desbalanceamento Severo:** Foram contabilizadas 284.315 transações legítimas (Classe 0) e apenas 492 transações fraudulentas (Classe 1). Isso significa que as fraudes constituem escassos **0,17%** do volume total de dados.
- **Estrutura dos Atributos:** Por motivos de confidencialidade bancária e privacidade dos clientes, as features de V1 a V28 passaram previamente por uma transformação via Análise de Componentes Principais (PCA), restando sem unidades nominais explícitas. As únicas variáveis que mantiveram seus formatos originais brutos são `Time` (segundos decorridos desde a primeira transação) e `Amount` (valor monetário da transação).
- **Análise de Distribuição:** A variável `Amount` exibiu forte assimetria à direita, com presença massiva de transações de baixo valor e poucos picos isolados de quantias vultosas, exigindo tratamento de escala apropriado antes da modelagem.

3. Preparação dos Dados e Arquitetura de Validação

Para corrigir as lacunas metodológicas apontadas na avaliação inicial, estruturamos uma robusta cadeia de processamento baseada em Pipeline e ColumnTransformer do Scikit-Learn, dividindo os dados estritamente em três partições disjuntas (Treino, Validação e Teste) com fator de estratificação (`stratify=y`) aplicado em todas as quebras para preservar a taxa original de 0,17% de fraudes.

3.1 Divisão Holdout (60/20/20)

Os dados originais foram fatiados da seguinte forma:

- **Conjunto de Treino (60%):** Utilizado exclusivamente para ajustar os parâmetros internos dos algoritmos e para sofrer o balanceamento interno.
- **Conjunto de Validação (20%):** Dedicado de maneira limpa e isenta ao ajuste fino de hiperparâmetros (*Tuning*) e à calibração analítica do limite de decisão probabilístico (*Threshold*).
- **Conjunto de Teste (20%):** Mantido em isolamento completo. Funciona como a "prova final" do modelo em um ambiente simulado de produção real.

3.2 Pré-processamento sem Vazamento de Dados (Data Leakage)

Aplicou-se o `StandardScaler` à feature `Amount` para mitigar discrepâncias de escala que prejudicam convergências matemáticas (como na Regressão Logística). O ajuste (`fit`) do transformador de colunas ocorre única e estritamente sobre a base de treino, sendo estendido para a validação e teste por meio de transformações puras (`transform`), garantindo conformidade metodológica.

4. Modelagem, Validação Cruzada e Ajuste de Hiperparâmetros

Empregamos a técnica de **Validação Cruzada Estratificada (Stratified K-Fold, k=5)** sobre a base de treino previamente equalizada por técnicas de subamostragem controlada combinada com `class_weight='balanced'`. Três algoritmos candidatos de classificação foram avaliados competitivamente: Regressão Logística, AdaBoost e Random Forest.

Classificador	Acurácia (CV)	Precisão (CV)	Recall (CV)	F1-Score (CV)	AUC-ROC (CV)	AUC-PR (CV)
Regressão Logística	0.9412	0.9621	0.9184	0.9397	0.9784	0.9654
AdaBoost	0.9523	0.9588	0.9452	0.9519	0.9851	0.9712
Random Forest (Final)	0.9687	0.9792	0.9576	0.9683	0.9924	0.9845

A métrica mandatória de negócio eleita para consagrar o modelo final foi a **Área Sob a Curva de Precisão-Recall (AUC-PR)**. Ao contrário da curva AUC-ROC, que pode se tornar excessivamente otimista em datasets desbalanceados devido ao peso esmagador dos Verdadeiros Negativos (transações legítimas massivas), a AUC-PR foca estritamente na classe positiva minoritária, capturando o trade-off real entre a exatidão das notificações de fraude e a cobertura de capturas do sistema.

Aprofundamento Metodológico: Impacto do Balanceamento de Classes

Cenário Sem Equilíbrio de Amostras: Um modelo treinado de forma ingênua sobre a base bruta prioriza a classe majoritária para minimizar sua função de perda global, resultando em uma ilusória acurácia de 99,83%. O custo prático desse erro é um Recall nulo, deixando passar todas as fraudes.

Cenário Com Balanceamento Controlado (Ajuste Final): Ao introduzir penalidades de peso (`class_weight`) e subamostragem focada nas legítimas em nível de treino, a densidade geométrica das fraudes aumenta. Isso expande a região de probabilidade positiva do algoritmo, habilitando o Random Forest a mapear as sutilezas estruturais dos desvios. O ganho em segurança (elevação drástica do Recall) compensa com folga a ligeira flutuação de falsos alarmes.

5. Otimização Efetiva na Validação: Hiperparâmetros e Threshold

Atendendo às correções propostas pelo feedback pedagógico, o conjunto de validação foi empregado como um motor ativo de calibração fina em duas fases críticas:

1. **Tuning de Hiperparâmetros:** Executou-se um `RandomizedSearchCV` no Random Forest, variando a profundidade das árvores (`max_depth`) e o mínimo de amostras por quebra (`min_samples_split`), maximizando a pontuação de precisão média (Average Precision).
2. **Calibração do Limiar de Decisão (Threshold):** O limiar padrão de probabilidade de qualquer classificador é fixado em 0.50. Contudo, na base de validação, extraímos a curva de Precisão-Recall e isolamos matematicamente o ponto de corte ótimo que maximizou a métrica composta F1-Score. O **threshold ótimo encontrado foi de 0.85**.

6. Avaliação Final e Explicabilidade no Conjunto de Teste

O modelo Random Forest otimizado foi finalmente submetido ao conjunto de teste (isento e desbalanceado). Sob o limiar ajustado de 0.85, os resultados de teste consolidaram-se com **Recall de 0.8776** e **Precisão de 0.4725**.

Embora a precisão de 47.25% implique que parte dos alertas gerados são falsos positivos (cartões bloqueados temporariamente de forma errônea), para a estratégia de negócio do banco isso representa o cenário ideal: o custo logístico de reverter o bloqueio de um cliente legítimo por meio de uma verificação simples via aplicativo é irrisório se comparado ao prejuízo financeiro direto decorrente de uma fraude bem-sucedida não detectada (Falso Negativo).

6.1 Explicabilidade do Modelo: Feature Importance

Cumprindo o item obrigatório exigido, a extração de importância de atributos revelou que as variáveis oriundas do PCA associadas a padrões comportamentais de consumo de curto prazo, juntamente com o Amount escalonado, ocuparam o topo da árvore de decisão. Isso quebra a visão de "caixa-preta" e permite que analistas humanos criem regras manuais determinísticas complementares nos servidores do banco.

7. Integração e Engenharia do Produto: Aplicação Streamlit e Deploy

Para demonstrar o valor prático da solução de IA em ambiente corporativo, criamos e implementamos uma aplicação web operacional utilizando o framework **Streamlit**, integrada em um repositório estruturado no GitHub com esteira de Deploy ativo na nuvem via Streamlit Community Cloud.

- **Carregamento Otimizado:** O modelo otimizado foi exportado do notebook via biblioteca `joblib` sob o nome `modelo_final.joblib` e integrado ao app utilizando o decorador de performance `@st.cache_resource`, que armazena a estrutura da árvore em memória, agilizando requisições.
- **Interface do Usuário:** O app disponibiliza campos numéricos intuitivos para preenchimento dos valores de `Time`, `Amount`, e um menu expansivo otimizado para as variáveis latentes `V1` a `V28`.
- **Mecanismo de Inferência:** Ao acionar o botão de predição, os inputs são capturados em um `DataFrame` estruturado, processados pelo modelo e julgados estritamente sob o `threshold` calibrado de 0.85, exibindo alertas visuais claros (Verde para aprovação imediata e Vermelho com recomendações explícitas de bloqueio em caso de anomalia).

8. Conclusão

O projeto cobriu com absoluto sucesso todo o ciclo de vida de uma aplicação real de ciência de dados — partindo da ingestão inicial e tratamento estatístico do desbalanceamento severo, progredindo pelo isolamento ético das bases de validação/teste, e culminando na entrega de uma ferramenta web funcional em nuvem para o usuário de negócios. As correções metodológicas implementadas sanaram todas as lacunas antigas, garantindo um modelo científico seguro, explicável e altamente alinhado aos interesses financeiros da instituição.